Sorting Algorithms: Complexities and Application Rules

Insertion Sort, Merge Sort, Bubble Sort, Selection Sort, Quick Sort

1. Time and Space Complexities

Algorithm	Best	Average	Worst	Space	Stable	In-place
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No
Bubble Sort	O(n) with early-exit flag	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n), O(n) worst	No	Yes

2. Worst-Case Trigger for Each Algorithm

Algorithm	What causes the worst case
Insertion Sort	Input sorted in reverse order
Merge Sort	No real worst case; always O(n log n)
Bubble Sort	Input sorted in reverse order
Selection Sort	Same for every input; no best or worst case
Quick Sort	Bad pivot every time (already sorted data with the first or last element as pivot)

3. Application Rules

Algorithm	Use it when	Reason	Keywords
Insertion Sort	Small data, nearly sorted data, data arriving one item at a time	Shifts only as far as needed; very low overhead	small input, nearly sorted, online, adaptive
Merge Sort	Large data, guaranteed performance needed, stability needed, linked lists, external sorting (disk, database)	Always splits and merges in linear time; reads data sequentially	guaranteed, stable, predictable, external, large dataset, linked list
Bubble Sort	Teaching, tiny inputs, checking if a list is already sorted	Simple, but does too many swaps and comparisons	educational, simple, adjacent swaps, inefficient
Selection Sort	Writes are expensive (flash, EEPROM) and data is small	At most n-1 swaps, the fewest of the simple sorts	minimum swaps, fewer writes, find minimum
Quick Sort	Large in-memory arrays where speed matters and stability does not	In-place, cache-friendly, small constants, fastest in practice	fastest in practice, in-place, pivot, partitioning, average case

4. Decision Rules for Scenario Questions

If the scenario says	Answer with
Worst-case guarantee, real-time, latency-sensitive, millisecond updates	Merge Sort
Equal keys must keep original order, ties, ranking	Merge Sort (stable)
Data too large for memory, database, file, disk	Merge Sort (external sorting)
Linked list	Merge Sort
Fastest average speed, memory is limited, order of ties does not matter	Quick Sort
Small dataset or almost sorted	Insertion Sort
Cost of writing to memory is high	Selection Sort
Just checking if sorted, or teaching	Bubble Sort

5. Limitations

Algorithm	Main limitations
Merge Sort	Needs O(n) extra memory, overhead on small arrays, not adaptive
Quick Sort	$O(n^2)$ worst case, not stable, recursion stack depth
Insertion Sort	$O(n^2)$ on large or reversed data
Bubble Sort	$O(n^2)$ almost always, too many swaps
Selection Sort	$O(n^2)$ always, not stable, not adaptive

F I N